Homework 07

Haoran Xu

2026-10-12

All work is in R. `MASS` supplies `lda`/`qda`; everything else is base R. Seed `432` is used wherever randomness enters (Questions 3 and 4).

Question 1 (LDA)

Solution

Part a. Priors are class proportions, centroids are class-wise column means, and the pooled covariance adds the two within-class scatter matrices and divides by $n - K$.

```
heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"]); y <- heart$chd     # famhist -> 1/2
n <- nrow(X); p <- ncol(X); cls <- sort(unique(y)); K <- length(cls)
pi_hat <- as.numeric(table(y)) / n                                    # length K
mu_hat <- t(sapply(cls, function(k) colMeans(X[y == k, ])))         # K x p
Sigma  <- Reduce(`+`, lapply(cls, function(k)
            crossprod(scale(X[y == k, ], scale = FALSE)))) / (n - K)  # p x p
c(n = n, p = p, K = K); round(pi_hat, 4)
#>   n   p   K
#> 462   9   2
#> [1] 0.6537 0.3463
round(mu_hat, 3)
#>         sbp tobacco   ldl adiposity famhist  typea obesity alcohol    age
#> [1,] 135.46   2.635 4.344    23.969   1.318 52.368  25.737  15.931 38.854
#> [2,] 143.74   5.525 5.488    28.120   1.600 54.494  26.623  19.145 50.294
round(Sigma[, 1:5], 2)                                                # first 5 columns shown
#>              sbp tobacco   ldl adiposity famhist
#> sbp       405.43   14.59  4.58     49.16    0.34
#> tobacco    14.59   19.24  0.76      7.54    0.02
#> ldl         4.58    0.76  4.00      6.03    0.09
#> adiposity  49.16    7.54  6.03     56.75    0.43
#> famhist     0.34    0.02  0.09      0.43    0.23
#> typea     -15.59   -2.06  0.34     -5.31    0.08
#> obesity    18.94    1.83  2.66     22.71    0.18
#> alcohol    64.40   20.52 -2.53     16.12    0.77
#> age        95.13   22.77  6.48     60.51    1.00
```

- $\hat{\pi} = (0.654, 0.346)$: 302 without and 160 with CHD.
- The CHD centroid is higher on every predictor, most clearly for `age`, `tobacco`, `ldl`, and `famhist`.
- $\hat{\Sigma}$ is 9×9; the scale differences (e.g. `sbp` variance 405 versus `ldl` variance 4) are harmless because the score uses $\hat{\Sigma}^{-1}$.

Part b. The score for class k is $\mathbf{x}_0^\top \hat{\Sigma}^{-1} \hat{\mu}_k - \frac{1}{2} \hat{\mu}_k^\top \hat{\Sigma}^{-1} \hat{\mu}_k + \log \hat{\pi}_k$; computed for all rows at once, $\mathbf{X} \hat{\Sigma}^{-1} \hat{\mathbf{M}}^\top$ is $n \times K$.

```r
Sinv  <- solve(Sigma)
lin   <- X %*% Sinv %*% t(mu_hat)                                  # n x K
quad  <- 0.5 * diag(mu_hat %*% Sinv %*% t(mu_hat))                 # length K
score <- sweep(lin, 2, quad) + rep(log(pi_hat), each = n)          # n x K
pred  <- cls[apply(score, 1, which.max)]
dim(score); table(observed = y, predicted = pred); mean(pred != y)
#> [1] 462   2
#>          predicted
#> observed   0   1
#>        0 258  44
#>        1  73  87
#> [1] 0.25325
```

- In-sample misclassification rate $117/462 = 0.253$.
- Errors are asymmetric: 73 of 160 CHD cases are called healthy, but only 44 of 302 healthy cases are called CHD, because the prior favours class 0 and the classes overlap.

Part c. `MASS::lda` with the same predictors and empirical priors.

```r
fit <- lda(X, grouping = y)                          # default prior = class proportions
pred_lda <- predict(fit)$class
table(observed = y, predicted = pred_lda)
#>          predicted
#> observed   0   1
#>        0 258  44
#>        1  73  87
post_mine <- exp(score - apply(score, 1, max)); post_mine <- post_mine / rowSums(post_mine)
c(same_label = mean(pred_lda == pred), max_posterior_gap = max(abs(post_mine -
 ↪  predict(fit)$posterior)))
#>       same_label max_posterior_gap
#>       1.0000e+00       1.2323e-14
```

- The confusion matrices are identical and the posteriors agree to 10^{-14}. `lda` works on a sphered scale and its `scaling` vector differs from $\hat{\Sigma}^{-1}\hat{\mu}_k$ by a constant, but the class decisions are the same function.

Question 2 (QDA and marginal screening)

Solution

Part a. Marginal variances are computed on the training pixels only; the 40 pixel positions with the largest variance are then reused in the test set.

```r
ztr <- as.matrix(read.csv(gzfile("data/zip-train.csv.gz"), header = FALSE))
zte <- as.matrix(read.csv(gzfile("data/zip-test.csv.gz"),  header = FALSE))
Xtr <- ztr[, -1]; ytr <- ztr[, 1]; Xte <- zte[, -1]; yte <- zte[, 1]
c(n_train = nrow(Xtr), n_test = nrow(Xte), p = ncol(Xtr))
#> n_train  n_test       p
#>    7291    2007     256
v   <- apply(Xtr, 2, var)
qda_top <- function(m) {
  sel <- order(v, decreasing = TRUE)[1:m]
  predict(qda(Xtr[, sel], grouping = ytr), Xte[, sel])$class
}
p40 <- qda_top(40)
table(observed = yte, predicted = p40); mean(p40 != yte)
#>          predicted
#> observed   0   1   2   3   4   5   6   7   8   9
#>        0 319   0  24   6   2   2   1   0   5   0
```

```
#>        1   0 238   5   0  11   0   7   0   3   0
#>        2   8   0 168   8   8   3   0   0   3   0
#>        3   5   0   4 128   0  14   0   0  15   0
#>        4   0   2  13   1 150   1   0   1   7  25
#>        5  12   0   3   5   1 131   0   0   6   2
#>        6   0   0   8   0   1   8 148   0   5   0
#>        7   1   0   6   1   6   0   0 124   2   7
#>        8   1   0  10  12   3   2   0   0 137   1
#>        9   0   0   1   2   7   1   1   4   9 152
#> [1] 0.15546
```

- Test misclassification rate 0.155 (312 of 2007).
- Largest confusions: 4 predicted as 9 (25), 0 as 2 (24), and 3 as 8 (15) or 5 (14).

Part b. Yes. With all 256 pixels `qda` stops with *rank deficiency in group 0*: some border pixels are constant -1 inside a class (two of them for digit 0), so the class covariance is singular and cannot be inverted. Screening keeps only pixels that vary; with 40 predictors each class has at least 542 training images (digit 8) to estimate $40 \cdot 41/2 = 820$ covariance entries, so all ten $\hat{\Sigma}_k$ are invertible and QDA runs. The error of 0.155 is usable but worse than the KNN results from Homework 5, because the screened pixels are chosen for variance across all digits, not for separating classes, and 40 out of 256 pixels discard most of the shape information.

Part c.

```
p10 <- qda_top(10); mean(p10 != yte)
#> [1] 0.3423
```

- The test misclassification rate more than doubles, to 0.342.
- Fewer predictors means less variance in estimating the ten $\hat{\Sigma}_k$ (only 55 entries each), but far more bias: ten pixels, almost all in the central column of the image, cannot separate digits that differ elsewhere (2/6/0, 4/9, 8/3). Here the information lost outweighs the estimation saved, so the bias-variance trade-off favours more pixels.
- Marginal variance is an unsupervised filter; a pixel can vary a lot without being related to the label, so the first 10 are not the 10 most useful.

Question 3 (Compare LDA and QDA)

Solution

Both classes have unit variances on X_1, X_2; the correlation is $+\rho$ in class 1 and $-\rho$ in class 2. I generate correlated pairs as $X_2 = \rho Z_1 + \sqrt{1-\rho^2} Z_2$, append eight $N(0,1)$ noise covariates, and use equal priors in both methods.

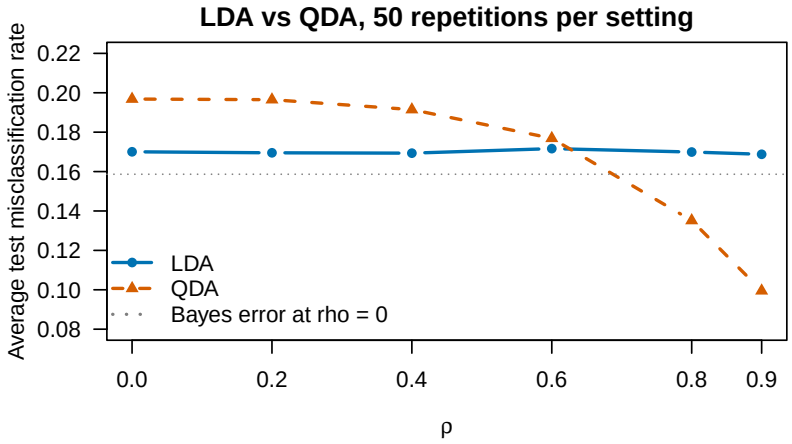
```
gen <- function(n_per, rho) {
  one <- function(mu1, r, lab) {
    z <- matrix(rnorm(n_per * 10), n_per, 10)
    z[, 2] <- r * z[, 1] + sqrt(1 - r^2) * z[, 2]; z[, 1] <- z[, 1] + mu1
    list(x = z, y = rep(lab, n_per))
  }
  a <- one(-1, rho, 1); b <- one(1, -rho, 2)
  list(x = rbind(a$x, b$x), y = c(a$y, b$y))
}
errs <- function(rho) {                          # same train/test for both methods
  tr <- gen(100, rho); te <- gen(1000, rho)
  l <- lda(tr$x, tr$y, prior = c(.5, .5)); q <- qda(tr$x, tr$y, prior = c(.5, .5))
  c(LDA = mean(predict(l, te$x)$class != te$y), QDA = mean(predict(q, te$x)$class != te$y))
}
rhos <- c(0, .2, .4, .6, .8, .9)
set.seed(432); one_run <- sapply(rhos, errs); colnames(one_run) <- rhos; one_run
#>          0   0.2   0.4    0.6    0.8   0.9
#> LDA 0.1505 0.165 0.166 0.1775 0.1695 0.161
```

```
#> QDA 0.1695 0.189 0.192 0.1760 0.1355 0.111
```

Part a. Single datasets (seed 432): LDA is better at $\rho \leq 0.4$, the two are about equal at 0.6, and QDA wins clearly at 0.8 and 0.9.

Part b. 50 repetitions per ρ, fresh training and test sets each time.

```
set.seed(432)
avg <- sapply(rhos, function(r) rowMeans(replicate(50, errs(r)))); colnames(avg) <- rhos
round(avg, 4)
#>          0    0.2    0.4    0.6    0.8    0.9
#> LDA 0.1700 0.1695 0.1694 0.1716 0.1699 0.1688
#> QDA 0.1968 0.1965 0.1915 0.1769 0.1352 0.0995
par(mar = c(4, 4, 2, 1), cex = 0.8)
matplot(rhos, t(avg), type = "b", pch = c(16, 17), lty = c(1, 2), lwd = 2,
        col = c("#0072B2", "#D55E00"), ylim = c(0.08, 0.22), las = 1, xaxt = "n",
        xlab = expression(rho), ylab = "Average test misclassification rate",
        main = "LDA vs QDA, 50 repetitions per setting")
axis(1, at = rhos); abline(h = pnorm(-1), lty = 3, col = "gray50")
legend("bottomleft", c("LDA", "QDA", "Bayes error at rho = 0"), col = c("#0072B2", "#D55E00",
 ↪  "gray50"),
       lty = c(1, 2, 3), pch = c(16, 17, NA), lwd = 2, bty = "n")
```



Part c.

- **Small ρ.** The two class covariances are nearly equal, so LDA's assumption holds and the Bayes boundary is close to the line $x_1 = 0$ (Bayes error $\Phi(-1) = 0.159$). LDA averages 0.170; QDA averages 0.197. Both are unbiased here, so the gap is pure variance: LDA estimates one 10×10 covariance (55 parameters) from 200 points, QDA estimates two from 100 points each, and the noisier $\hat{\Sigma}_k^{-1}$ produce a wobblier boundary.
- **Large ρ.** The covariances now differ a lot ($+0.9$ versus -0.9), the true boundary is quadratic, and LDA's single pooled covariance (whose off-diagonal averages to about zero) is badly biased: its error stays flat near 0.17 no matter how separable the classes become. QDA follows the true boundary and reaches 0.100 at $\rho = 0.9$. The crossover is near $\rho = 0.6$, where QDA's bias advantage first outweighs its extra variance.
- **Noise covariates.** They add no signal but both methods still estimate their means and (co)variances. For LDA that is 8 extra mean differences and 44 extra covariance entries; for QDA, double the covariance entries. Each adds estimation noise to the discriminant direction, which is why LDA sits above the Bayes error even at $\rho = 0$ and why QDA is penalised more. With more noise dimensions (or less training data) the crossover would move to larger ρ.

Question 4 (Create and use an LDA/QDA analysis skill)

Solution

Part a. The skill is `stat432-lda-qda` (full text in Appendix B, 3,980 bytes). It fixes the protocol (reserve a stratified test set, build stratified folds once, candidates limited to LDA, QDA, LDA/QDA on m principal components, and a posterior blend $w\cdot$LDA $+(1-w)\cdot$QDA; all preprocessing fitted inside each fold; selection by CV misclassification rate with a simplicity tie-break; one final test evaluation), the report structure (five fixed headings, one table, one figure, at most one page), and the style (plain language, no claims when the gap is within one SE). Nothing in it names a dataset, a variable, or a result.

Part b. The agent (Claude Code) was given the data path, the response `chd`, the nine predictors with `famhist` coded 1/2, and seed `432`. It wrote `q4-analysis.R` and the fragment `q4-report.qmd`, included on the next page with one edit by me: the figure height was reduced from 2.6 to 2.2 inches so the page also holds the appendix heading. Summary of what it did: 25% stratified test split (116 test, 346 training), ten stratified folds reused for every candidate; standardisation and PCA fitted on the training part of each fold; plain LDA and QDA, PCA-LDA and PCA-QDA for $m = 1, \ldots, 9$, and a blend of the CV-best member of each family for $w \in \{0, 0.05, \ldots, 1\}$; the chosen model refit on all 346 training rows and scored once on the test set.

Part c. I checked the script line by line for leakage (none: the test rows are only touched in the final block, and fold posteriors are computed before any blend weight is chosen) and reran it to confirm the numbers in the report. The agent reported four ambiguities in my skill, all visible from the training/CV stage alone, and I revised the skill accordingly: (i) the tie-break did not say where raw-predictor models sit relative to PCA models or how to break tied blend weights, so the rule now reads "fewer components (raw $= p$), then no PCA, then LDA, then least blending"; (ii) the SE was undefined, now sd(fold rates)$/\sqrt{K}$; (iii) the 450-word budget could not fit on one page with the required table and figure, so it is now 320 words and captions are forbidden; (iv) "one figure" conflicted with two tuning axes, so side-by-side panels are allowed. The agent's choices already matched the clarified rules, so rerunning gives the same report. My own reading of its results: the blend's CV advantage (0.240 versus 0.246 for PCA-LDA and 0.263 for plain LDA) is well inside one SE, so the honest conclusion is that the SAheart data do not reward anything beyond LDA; the one test evaluation (0.259) agrees with the CV estimates.

Explicit invocation. The file lives in `~/.claude/skills/stat432-lda-qda/SKILL.md` (a copy is in `homework/week-07/stat432-lda-qda/`). Its `description` field tells Claude Code to use it only when the student explicitly names `stat432-lda-qda` and supplies the data, response, and predictors, and never to trigger it automatically; the skill therefore never loads unless I call it by name. The prompt used (edited only to shorten paths):

```
Use the skill `stat432-lda-qda`: read ~/.claude/skills/stat432-lda-qda/SKILL.md
first and follow it exactly. Dataset: homework/week-07/data/saheart.csv.
Response: `chd` (0/1). Predictors: the other nine columns, with `famhist`
coded 1 = Absent, 2 = Present. Random seed: 432. Use R only, with MASS::lda,
MASS::qda, prcomp, and knitr::kable. Seek the best predictive performance
within the LDA/QDA methods the skill allows, and include ordinary LDA and QDA
as comparisons. Deliverables: q4-analysis.R (reproduces everything from the
seed) and q4-report.qmd (the one-page report as a Quarto fragment: no YAML,
all chunks echo: false, level-4 headings, one base-R figure 5 x 2.6 in at
70% width, one knitr::kable table, confusion matrix inline in prose, at most
450 words). Run the script with Rscript to verify it and fill every number
from the actual output. Do not open any file named solutions-07.qmd.
```

Appendix A: one-page analysis report produced by the skill

Data and setup

The data contain 462 men and nine predictors. Coronary heart disease is coded 1 for the 160 cases (34.6%) and 0 for the 302 controls; family history was coded 1 = Absent, 2 = Present. With seed 432, a stratified 25% test set of 116 observations (76 controls, 40 cases) was reserved, leaving 346 (226, 120) for training; ten stratified folds were built once and reused for every candidate.

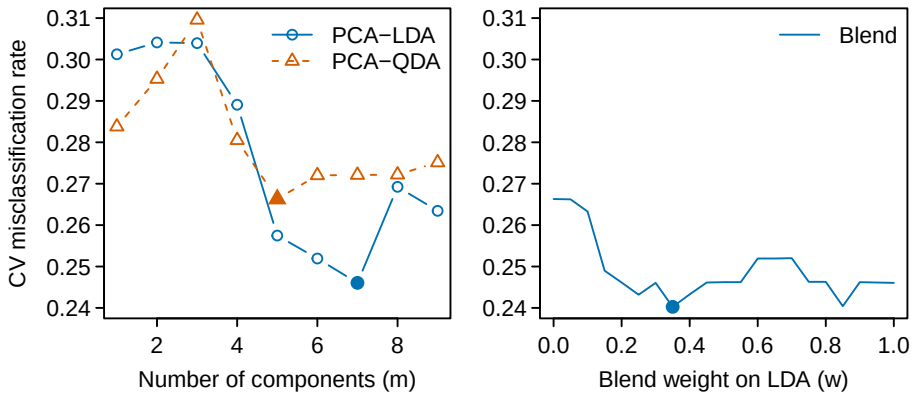
Candidate models

Plain LDA and QDA with empirical priors were the baselines. PCA-LDA and PCA-QDA used m = 1, ..., 9 principal components of the standardized predictors, standardized and rotated using each fold's training part. A blend, posterior w × LDA + (1 − w) × QDA for w from 0 to 1 by 0.05, combined the best LDA- and QDA-type candidates.

Cross-validation results

The table gives the ten-fold CV misclassification rate (SE) for the best setting per family; the figure shows the tuning paths, with chosen settings filled.

Candidate	Setting	CV error	SE
LDA	all 9 predictors	0.263	0.019
QDA	all 9 predictors	0.275	0.019
PCA-LDA	m = 7	0.246	0.026
PCA-QDA	m = 5	0.266	0.024
Blend (LDA / QDA)	w = 0.35	0.240	0.025



Selected model and test error

The blend with w = 0.35 of PCA-LDA with seven components (92.4% of the variance) and PCA-QDA with five (77.0%) had the lowest CV misclassification rate, 0.240 (SE 0.025). Refit on the 346 training observations (priors 0.653 for no CHD, 0.347 for CHD), its test misclassification rate was 0.259 (30 of 116). The confusion matrix (observed rows, predicted columns) was: no CHD, 66 predicted no CHD and 10 predicted CHD; CHD, 20 and 20 (sensitivity 0.500, specificity 0.868).

Findings and limitations

- PCA-LDA with seven components (0.246) edged plain LDA (0.263) and the blend gained 0.006 more, but all gaps are within one CV SE (about 0.025), so no candidate is clearly better.
- The test rate, 0.259, agrees with CV within one SE; the model misses half the CHD cases.
- Limitations: small samples (test SE near 0.04), dependence on one split and fold assignment, non-Gaussian predictors (skewed tobacco and alcohol, binary family history), and an untuned 0.5 threshold.

Appendix B: final `SKILL.md` (3,980 bytes)

```
---
name: stat432-lda-qda
description: Run a complete LDA/QDA classification analysis (R or Python) and write a
 ↪  one-page statistical report. Use only when the student explicitly names "stat432-lda-qda"
 ↪  and supplies a dataset, response, and predictors; never trigger it automatically.
---

# STAT 432 LDA/QDA analysis

Goal: find the best predictive classifier among LDA, QDA, and their
combinations, using honest validation, and report it on one printed page.

## Inputs (the request must supply; ask if missing)

Data file, response variable, predictors and any recoding, random seed.
Optional: test fraction (default 25%), number of folds (default 10).
Never hard-code names or numbers from one dataset into this skill.

## Analysis protocol

1. **Reserve a test set** first: one stratified random split with the seed.
   Touch the test set exactly once, at the very end.
2. **Folds.** Build stratified K-fold indices on the training set once and
   reuse the same folds for every candidate and every setting.
3. **Candidates** (all based on LDA/QDA only; always include the first two):
   - LDA with empirical priors; QDA with empirical priors.
   - LDA and QDA on the first m principal components of the standardized
     predictors, m over a grid from 1 to p.
   - Blend: posterior = w * LDA + (1 - w) * QDA, w on a grid from 0 to 1,
     applied to the best LDA-type and best QDA-type candidate; classify by the
     larger blended posterior.
4. **Preprocessing inside folds.** Centering, scaling, and PCA are estimated
   on the training part of each fold only, then applied to the held-out part.
   The same rule applies to the final refit on all training observations.
5. **Selection.** Score by CV misclassification rate (average over folds)
   with SE = sd(fold rates)/sqrt(K). Choose the lowest rate; on ties prefer
   the simpler model: fewer components (raw predictors count as p), then no
   PCA, then LDA over QDA, then the weight nearest 0 or 1 (least blending).
6. **Final fit.** Refit the chosen model and its preprocessing on all training
   observations. Compute the test misclassification rate once, plus the test
   confusion matrix (observed rows, predicted columns).
7. **Sanity checks.** Confirm class counts in every fold, no missing values,
   and that no step reads test rows before step 6. Report the seed.

## Report structure (strict)

At most one printed page, at most 320 words of prose, with exactly one
figure and one table, neither captioned (a caption floats the figure).
Use these headings, in this order:

1. **Data and setup.** n, p, class balance, response coding, split sizes,
   folds, seed. Two or three sentences.
2. **Candidate models.** One sentence each on what is tuned and why.
3. **Cross-validation results.** The one table: rows are candidates (at most
   8), columns are setting, CV error, SE. The one figure: CV error versus the
```

```
   tuning value, one line per method, the chosen setting marked; use side by
   side panels when methods have different tuning axes. Figure about
   5 x 2.6 inches, labeled axes, legend without a box, colorblind-safe
   colors, distinct line types.
4. **Selected model and test error.** Name the model and setting; give the
   numerical fitting results (class priors, number of components or weight,
   test misclassification rate, test confusion matrix inline as counts).
5. **Findings and limitations.** Three to five bullets: what drove the choice,
   whether differences exceed CV standard errors, what the test error means,
   and limitations (sample size, split variability, Gaussian assumptions,
   anything not tuned).

## Style

- Plain language, past tense, no code in the report prose. Round rates to
  three decimals. Say "misclassification rate", not "accuracy".
- Do not claim one method is better when the CV gap is within one SE.
- Deliver two files: an analysis script that reproduces everything from the
  seed, and the report as a Quarto fragment whose code chunks are hidden.
```